Chapter 1

Query processing and Optimization

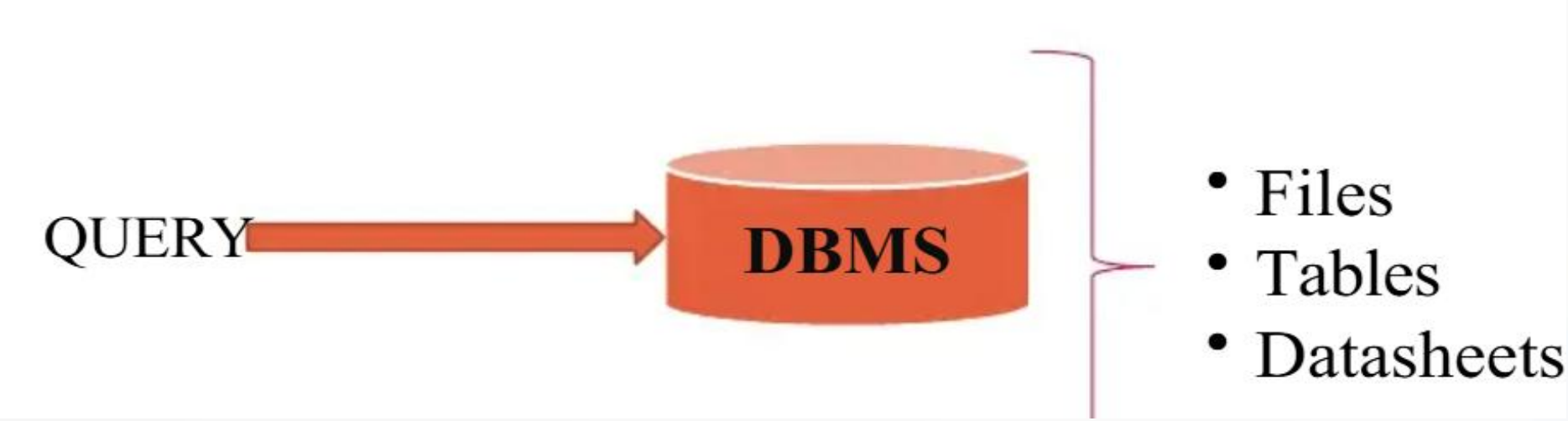
Contents

1. **Query Processing**
   - Steps of Processing
2. **Methods of Optimization**
   - Heuristic (Logical Transformations)
     - ✓ Transformation Rule
     - ✓ Heuristic Optimization Guidelines
   - Cost Based (Physical Execution Costs)
     - Data Storage/Access Refresher

Cont----------

What Query?

A query is a request for data or information from a database table or combination of tables.



SQL

SELECT,FROM, WHERE

1ST FROM

2ND WHERE

3RD SELECT

Cont--

Student-info

ID	NAME	SEM
1	A	5
2	B	6
3	C	1
4	D	1

1. **Fetch the details 0f 1st semester students.**

- SELECT * FROM Student-info WHERE SEM =1
- This query is in high level language and will be converted into low level language (Relational Algebra)

❖ **Relational Algebra**

- ✓ Symbols in Relational algebra
- ✓ **π--**Projection (Selection part of query, SELECT)
- ✓ **σ**--Selection (condition part of query , WHERE)
- ✓ ⋈ **--**Go -Used to join two tables
- ✓ Now, the above query in relational algebra
- ✓ **π ID,NAME,SEM(σ SEM=1 (Student-info)**

Cont--

What is Query Processing?

- ✓ Steps required to transfomation high level SQL query into a correct and efficient strategy for execution and retrieval .

Query Optimization?

- ✓ The activity of choosing a single efficient execution strategy (from hundreds) as determined by database catalog statistics.
- ✓ Which relational algebra expression, equivalent to the given query,will lead to the most efficient solution plan?
- ✓ For each algebraic operator, what algorithm (of several available) do we use to compute that operator?
- ✓ How do operations pass data (main memory buffer, disk buffer,...)?
- ✓ Will this plan minimize resource usage? (CPU/Response Time/Disk)

Cont----------

- Staff

S_ID	FName	LName	Position	Salary	branchNo
001	Tola	Waqjira	Chasher	5000	777
002	Habiba	Ahmed	Manager	8000	999
003	Abdi	Jiregna	Casher	6000	555
004	Lelise	Boru	DBA	7000	999

- Branch

branchNo	BranchName	City
999	Batu	Batu
777	Abba Gada	Adama
555	Bishoftu	Bishoftu

Consider these tables for next slides

Cont-----

Example:Identify all managers who work in Adama city

```
SELECT * FROM Staff s, Branch b
WHERE s.branchNo = b.branchNo AND s.position ='Manager' AND
        b.citv = 'Adama'
```

✓ Results in these equivalent relational algebra statements

A. σ(position=’manager’)^(city=’Adama’)^(Staff.branchNo=Branch.branchNo) (**Staff X Branch**)

B. σ(position=’manager’)^(city=’Adama’) (**Staff**) ⋈ Staff.branchNo=Branch.branchNo **Branch**)

C. [σ(position=’manager’)(**Staff**)] ⋈ Staff.branchNo=Branch.branchNo [(city=’Adama’) **Branch**)]

Assume:

- 1000 tuples in ***Staff***
- 50 Managers
- 50 tuples in ***Branch***
- 5 Adama branches
- No indexes or sort keys
- All temporary results are written back to disk (memory is small)
- Tuples are accessed one at a time (not in blocks)

Cont---

Query A

σ(position='manager')^(city='Adama')^(Staff.branchNo=Branch.branchNo) (**Staff X Branch**)

- ✓ Requires (1000+50) disk accesses to read from Staff and Branch relations
- ✓ Creates temporary relation of Cartesian Product (1000*50) tuples
- ✓ Requires (1000*50) disk access to read in temporary relation and test predicate
- ✓ **Total Work= (1000+50) + 2*(1000*50) = 101,050 I/O operations**

what is the prediction of Query A?

Bad

Query B

σ(position='manager')^(city='Adama') (**Staff** ⋈ Staff.branchNo=Branch.branchNo **Branch**)

- ✓ Again requires (1000+50) disk accesses to read from Staff and Branch
- ✓ Joins Staff and Branch on branchNo with 1000 tuples (1 staff: 1 branch)
- ✓ Requires (1000) disk access to read in joined relation and check predicate
- ✓ **Total work = (1000+50) + 2*(1000) = 3050 I/O operations**

what is the prediction of Query B?

Better

3300 % Improvement over Query A

Cont------

Query C

[σ(position='manager')(^{Staff})] ⋈ Staff.branchNo=Branch.branchNo [(city='Adama') ^{Branch})]

- ✓ Read ***Staff*** relation to determine 'Managers' (1000 reads)
  - ❖ Create 50 tuple relation (50 writes)
- ✓ Read ***Branch*** relation to determine 'Adama' branches(50 reads)
  - ❖ Create 5 tuple relation (5 writes)
- ✓ Join reduced relation and check predicate (50 + 5 read.)
- ✓ **Total work = 1000 + 2*(50) + 5 + (50 + 5) = 1160 I/O oprations**

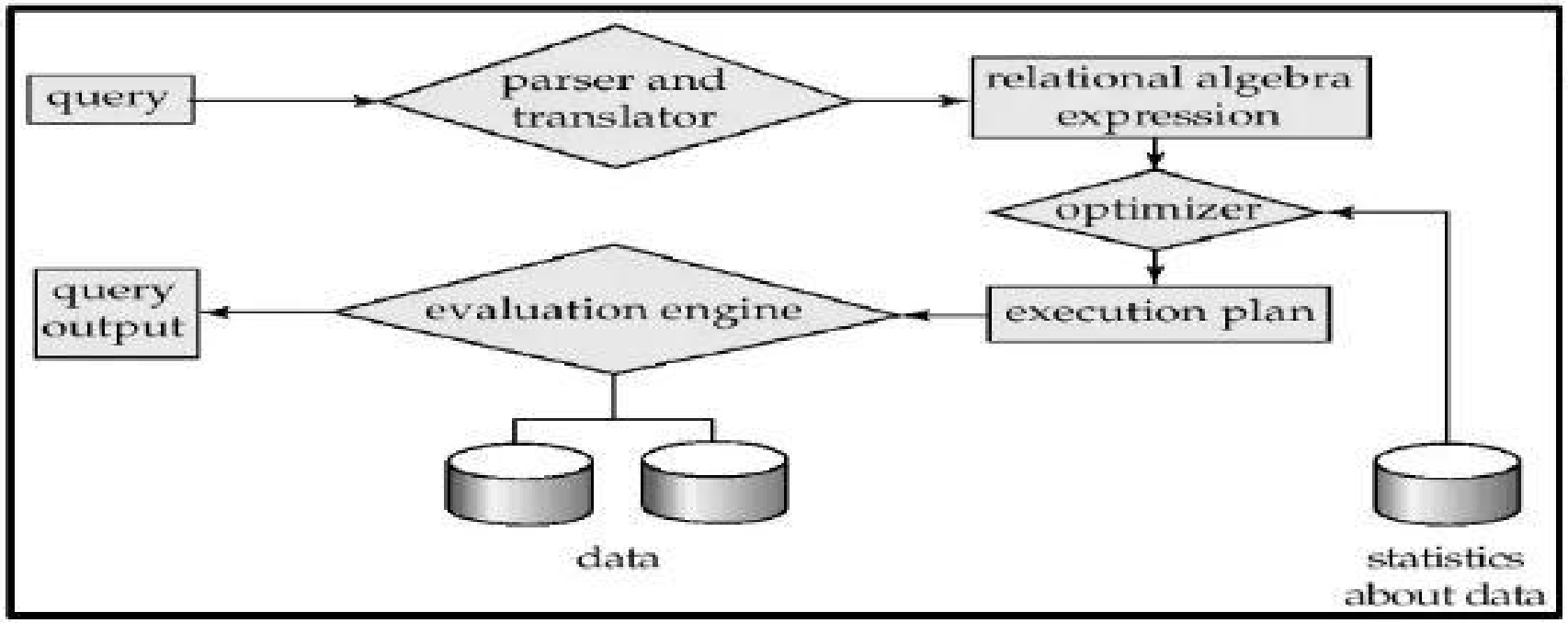
what is the prediction of Query C?

Best

- ✓ 8700% **Improvement over Query A**

cont------

Query Processing Steps



- Processing can be divided into:
  - Decomposition,
  - Optimization,
  - Execution and
  - Code generation

1. Query Decomposition

- ✓ It is the process of transforming a high level query into a relational algebra query
- ✓ to check that the query is syntactically and semantically correct.
- ✓ It consists of parsing and validation.

Typical stages in querydecomposition are:

a. Analysis

- ✓ lexical and syntactical analysis of the query(correctness) based on attributes,data type and etc.
- ✓ Query tree will be built for the query containing leaf node for base relations, one or many non-leaf nodes for relations produced by relational algebra operations.
- ✓ root node for the result of the query.
- ✓ Sequence of operation is from the leaves to the root.

SELECT* FROM Catalog c, Author a Where a.authorid = c.authorid AND c.price>200 AND a.country= 'USA')

b. Normalization

- ✓ Convert the query into a normalized form.
- ✓ The predicate where will be converted to conjunctive (^)or disjunctive (v) normal form

Cont------

c. Semantic Analysis

- ✓ To reject normalized queries that are not correctly formulated or contradictory.
- ✓ Incorrect: if components do not contribute to generate result.

  Contradictory: if the predicate can not be satisfied by any tuple.

example

(catalog ="BS" /\ catalog= "CS") since a given book can only be classified in either of the category at a time

d. Simplification

- ✓ To detect redundant qualifications, eliminate common sub- expressions.
- ✓ Transform the query to a semantically equivalent.

2. Query Optimization

- Everyone wants the performance of their database to be optimal.
- In particular, there is often a requirement for a specific query or object that is query based, to ***run faster.***
- Problem of query optimization is to find the sequence of steps that produces the answer to user request in the most efficient manner , given the database structure.
- The performance of a query is affected by the tables or queries that underlies the query and by the complexity of the query.
- Given a request for data manipulation or retrieval, an optimizer will choose an optimal plan for evaluating the request from among the various alternative strategies. i.e. there are many ways (access paths) for accessing desired file/record.
- hence ,DBMS is responsible to pick the best execution strategy based on various considerations(***Least amount of I/O and CPU resources.)***

Cont---

- Example: Consider relations ***r(AB)*** and ***s(CD)***. We require ***r X s.***
- Method 1 :
  a. Load next record of ***r*** in RAM.
  b. Load all records of ***s***, one at a time and concatenate with ***r***:
  c. All records of ***r*** concatenated?
     - NO: goto a.
     - YES: exit (the result in RAM or on disk).
  - Perfomiance: Too many accesses.
- Method 2: Improvement
  a. Load as many blocks of ***r*** as possible leaving room/or one block of ***s***.
  b. Run through the ***s*** file completely one block at a time.
- Performance: Reduces the number of times ***s*** blocks are loaded by a factor of equal to the number of ***r*** records than can fit in main memory.
- Considerations during query Optimization:
  ✓ Narrow down intermediate result sets quickly. ***SELECT*** and ***PROJECTION*** before ***JOIN***
  ✓ Use access structures (indexes)

Transformation Rules for Relational Algebra

Transformation Rules for Relational Algebra

1. Cascade of SELECTION: conjunctive SELECTION Operations can cascade into individual Selection Operations and Vice Versa

$$\sigma_{(c1 \wedge c2 \wedge c3)}(R) = \sigma_{c1}(\sigma_{c2}(\sigma_{c3}(R)))$$ where c_i is a predicate

2. Commutativity of SELECTION operations

$$\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R))$$ where c_i is a predicate

3. Cascade of PROJECTION: in the sequence of PROJECTION Operations, only the last in the sequence is required

$$\pi_{L1}\ \pi_{L2}\ \pi_{L3}\ \pi_{L4}(R) = \pi_{L1}(R)$$

4. Commutativity of SELECTION with PROJECTION and Vise Versa
   a. If the predicate $c1$ involves only the attributes in the projection list ($L1$), then the selection and projection operations commute

$$\pi_{L1}(\sigma_{c1}(R)) = \sigma_{c1}\ (\pi_{L1}(R))$$

Cont----

5. Commutativity of THETA JOIN/Cartesian Product
   R X S is equivalent to S X R
   Also holds for Equi-Join and Natural-Join

$$(R \bowtie_{c1} S) = (S \bowtie_{c1} R) \text{ or } (R X S) = (S X R)$$

6. Commutativity of SELECTION with THETA JOIN
   a. If the predicate $c1$ involves only attributes of one of the relations (R) being joined, then the Selection and Join operations commute

$$\sigma_{c1} (R \bowtie_{c} S) = (\sigma_{c1} (R)) \bowtie_{c} S)$$

   b. If the predicate is in the form $c1 \wedge c2$ and $c1$ involves only attributes of R and $c2$ involves only attributes of S, then the Selection and Theta Join operations commute

$$\sigma_{c1 \wedge c2} (R \bowtie_{c} S) = (\sigma_{c1} (R)) \bowtie_{c} (\sigma_{c2} S))$$

Cont---

7. Commutativity of PROJECTION and THETA JOIN

   If the projection list is of the form $L1, L2$, where $L1$ involves only attributes of R and $L2$ involves only attributes of S being joined and the predicate c involves only attributes in the projection list, then the SELECTION and JOIN operations commute

$$\pi_{L1,L2}(R \bowtie_c S) = (\pi_{L1}(R)) \bowtie_c (\pi_{L2} S))$$

8. Commutativity of the Set Operations: UNION and INTERSECTION but not SET DIFFERENCE

$$R \cap S = S \cap R \text{ and } R \cup S = S \cup R$$

9. Associativity of the THETA JOIN, CARTESIAN PRODUCT, UNION and INTERSECTION.

$$(R \theta S) \theta T = R \theta (S \theta T)$$ where θ is one of the operations

10. Commuting SELECTION with SET OPERATIONS

$$\sigma_c(R \theta S) = (\sigma_c(R) \ \theta \sigma_c(S))$$ where θ is one of the operations

11. Commuting PROJECTION with UNION

$$\pi_{L1}(S \cup R) = \pi_{L1}(S) \cup \pi_{L1}(R)$$

Approaches to Query Optimization

A. Heuristic Approach

- Uses the knowledge of the characteristics of the relational algebra operations.
- the relationship between the operators to optimize the query.
- Thus the heuristic approach of optimization will make use of:
  - ***Properties of individual operators***
  - ***Association between operators***

Query Tree:

- a graphical representation of the operators, relations, attributes and processing sequence

during query processing. It is composed of three main parts:

- A tree data structure that corresponds to a relational algebra expression.
  a) The Leafs: the base relations used for processing the query/ extracting the required information
  b) The Root: the final result/relation as an out put based on the operation on the relations used for query processing
  c) Nodes: intermediate results or relations before reaching the final result.

Sequence of execution of operation in a query tree will start from the leaves and continues to the intermediate nodes and ends at the root.

Cont---

Heuristic Approach will be implemented by using the above transformation rules in the following sequence or steps.

Sequence for Applying Transformation Rules

1. Use
   Rule-1➔ Cascade SELECTION
2. Use
   Rule-2: Commutativity of SELECTION
   Rule-4: Commuting SELECTION with PROJECTION
   Rule-6: Commuting SELECTION with JOIN and CARTESIAN
   Rule-10: commuting SELECTION with SET OPERATIONS
3. Use
   Rule-9: Associativity of Binary Operations (JOIN, CARTESIAN, UNION and INTERSECTION). Rearrange nodes by making the most restrictive operations to be performed first (moving it as far down the tree as possible)
4. Perform Cartesian Operations with the subsequent Selection Operation
5. Use
   Rule-3: Cascade of PROJECTION
   Rule-4: Commuting PROJECTION with SELECTION
   Rule-7: Commuting PROJECTION with JOIN and CARTESIAN
   Rule-11: commuting PROJECTION with UNION

Using Heuristics in Query Optimization

- ***Process for hetu·istics optimization***
  - The parser of a high-level query generates an initial internal representation;
  - Apply heuristics rules to optimize the internal representation.
  - A query execution plan is generated to execute groups of operations based on the access paths available on the files involved in the query.
- ***The main heuristic*** is to apply first the operations that reduce the size of intermediate results.
  - E.g. Apply ***SELECT*** and ***PROJECT*** operations before applying the ***JOIN*** or other binary operations.

Cont---

Query block:

- The basic unit that can be translated into the algebraic operators and optimized.
- Contains a single select-from-where expression, as well as group by and having clause if these are part of the block.

Nested queries

- Within a query are identified as separate query blocks.

Query graph

- ✓ A graph data structure that corresponds to a relational calculus expressrnn.

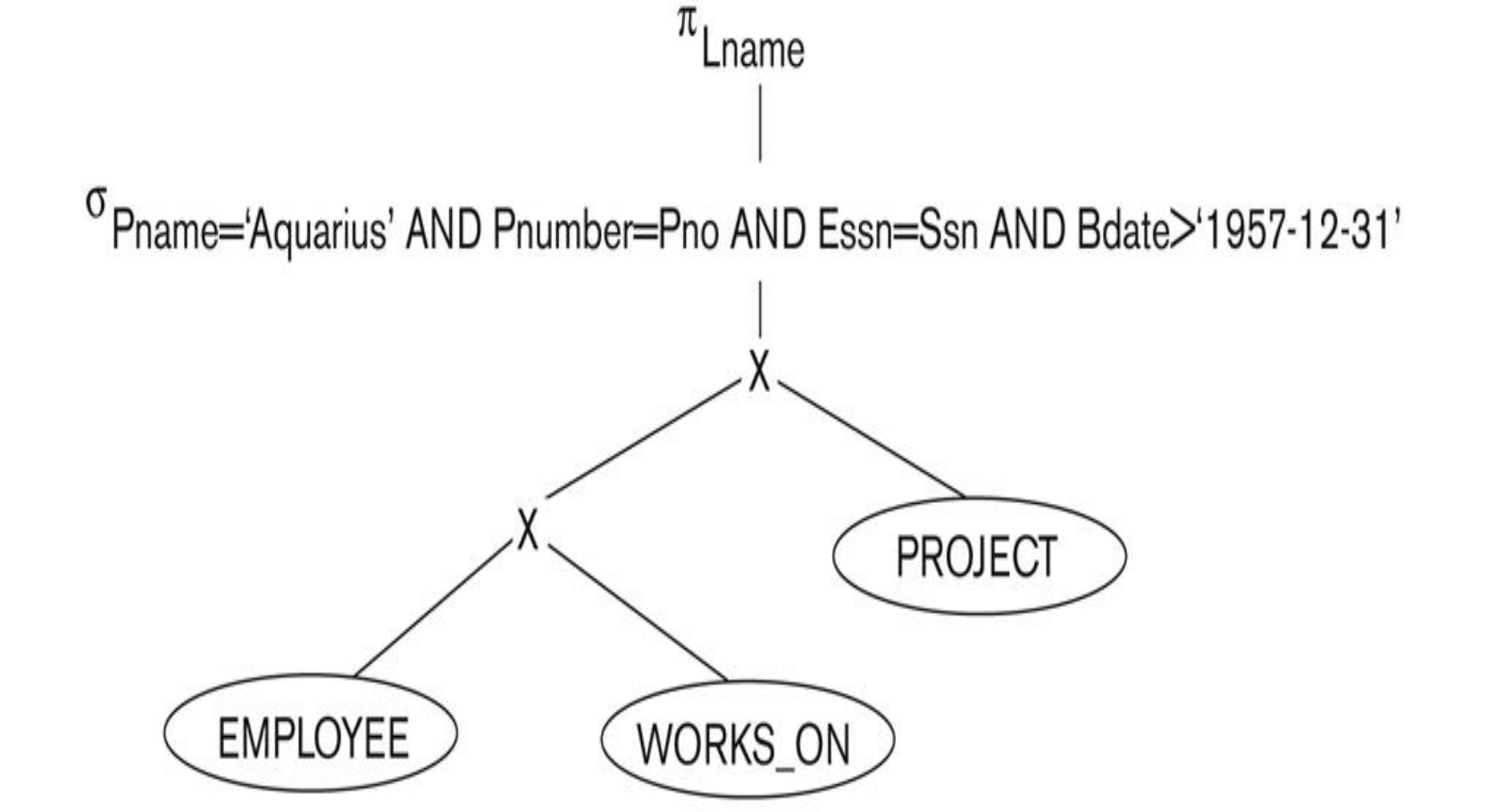
Example:

```
SELECT      LNAME
    FROM     EMPLOYEE, WORKS_ON, PROJECT
    WHERE  PNAME = 'AQUARIUS' AND
     PNMUBER=PNO AND ESSN=SSN
    AND BDATE > '1957-12-31';
```

Cont---

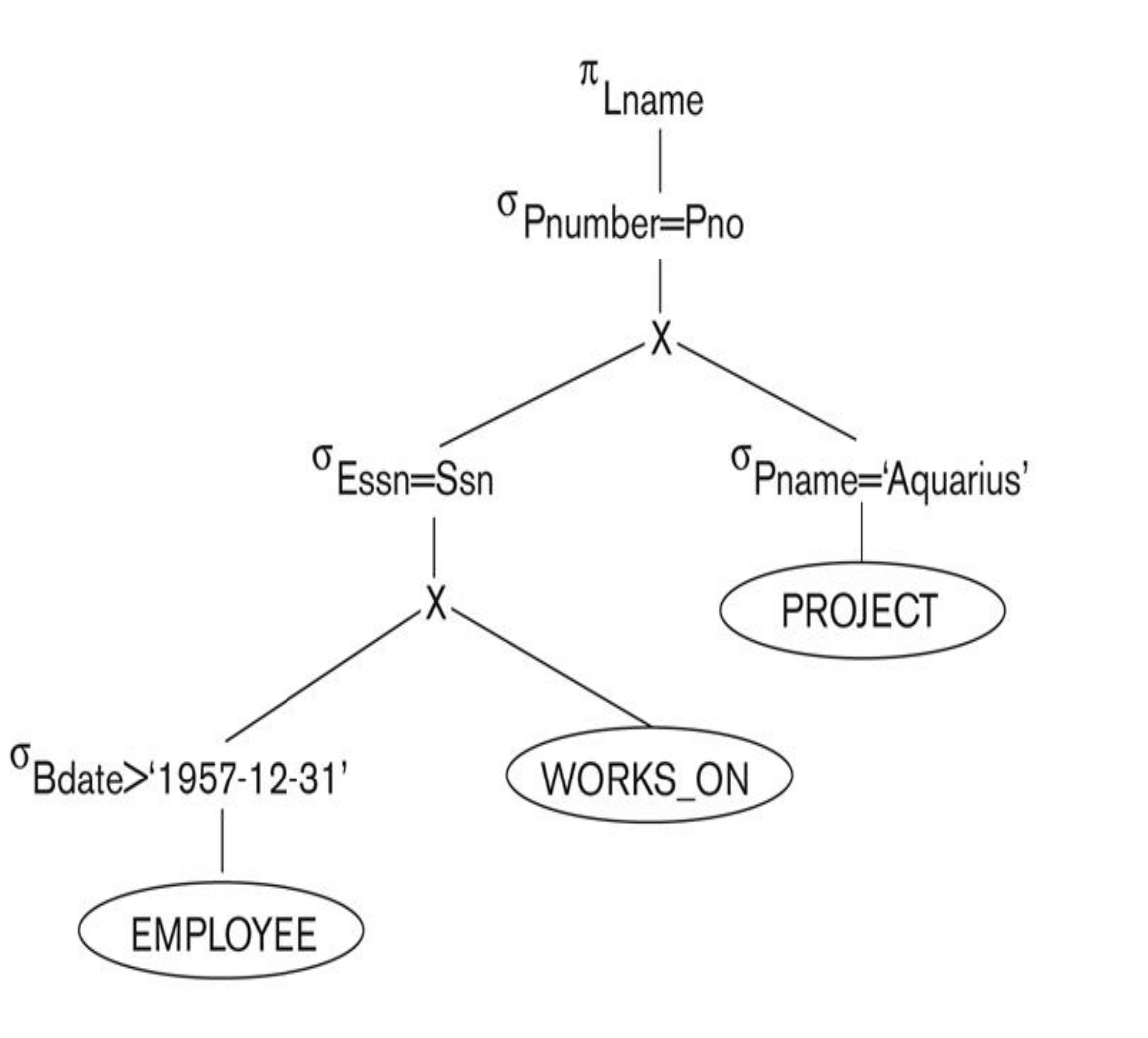
Steps in converting a query tree during heuristic optimization:

Initial query tree for the query Q on slide 18



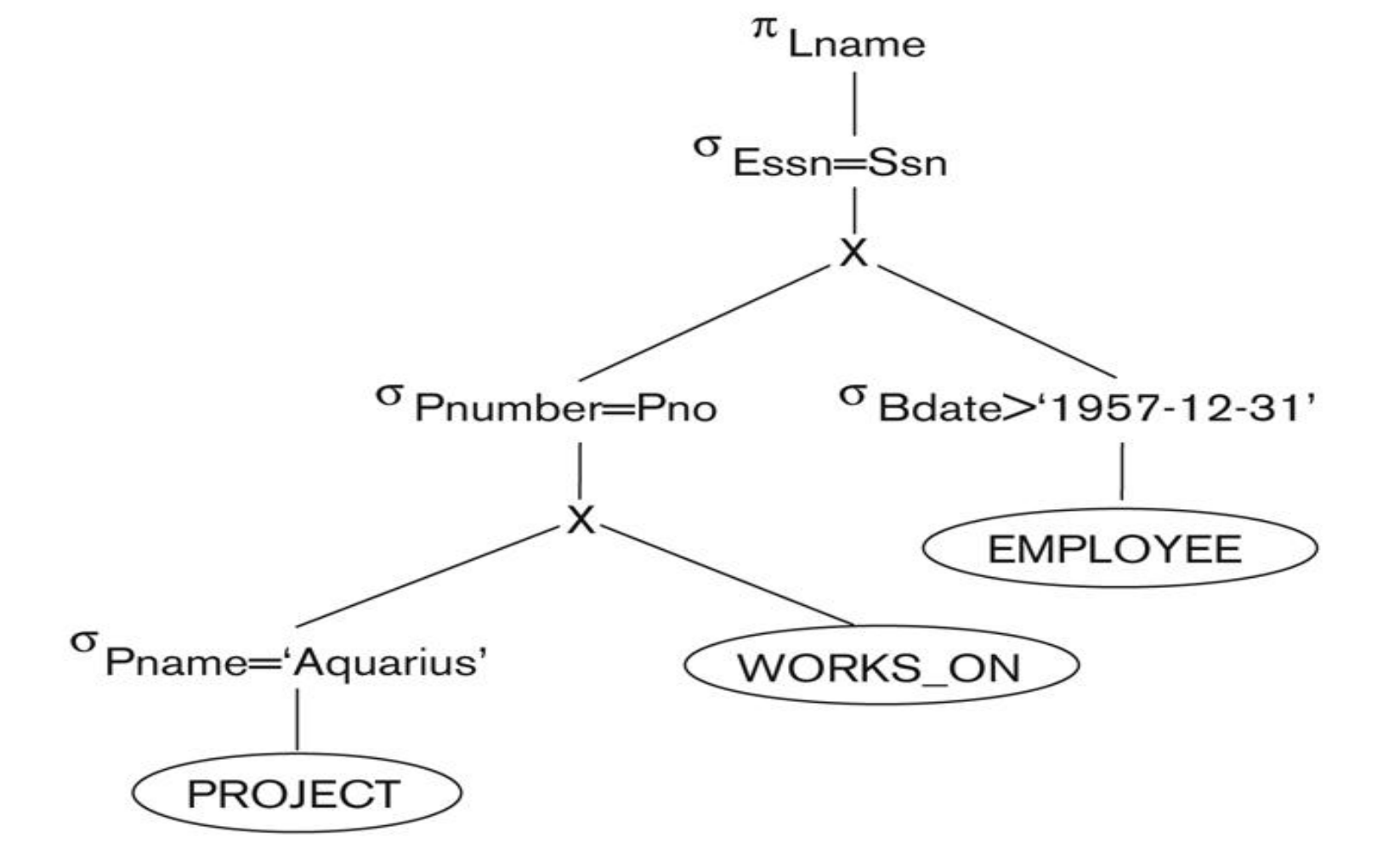
Cont--

Moving the select operation down the tree



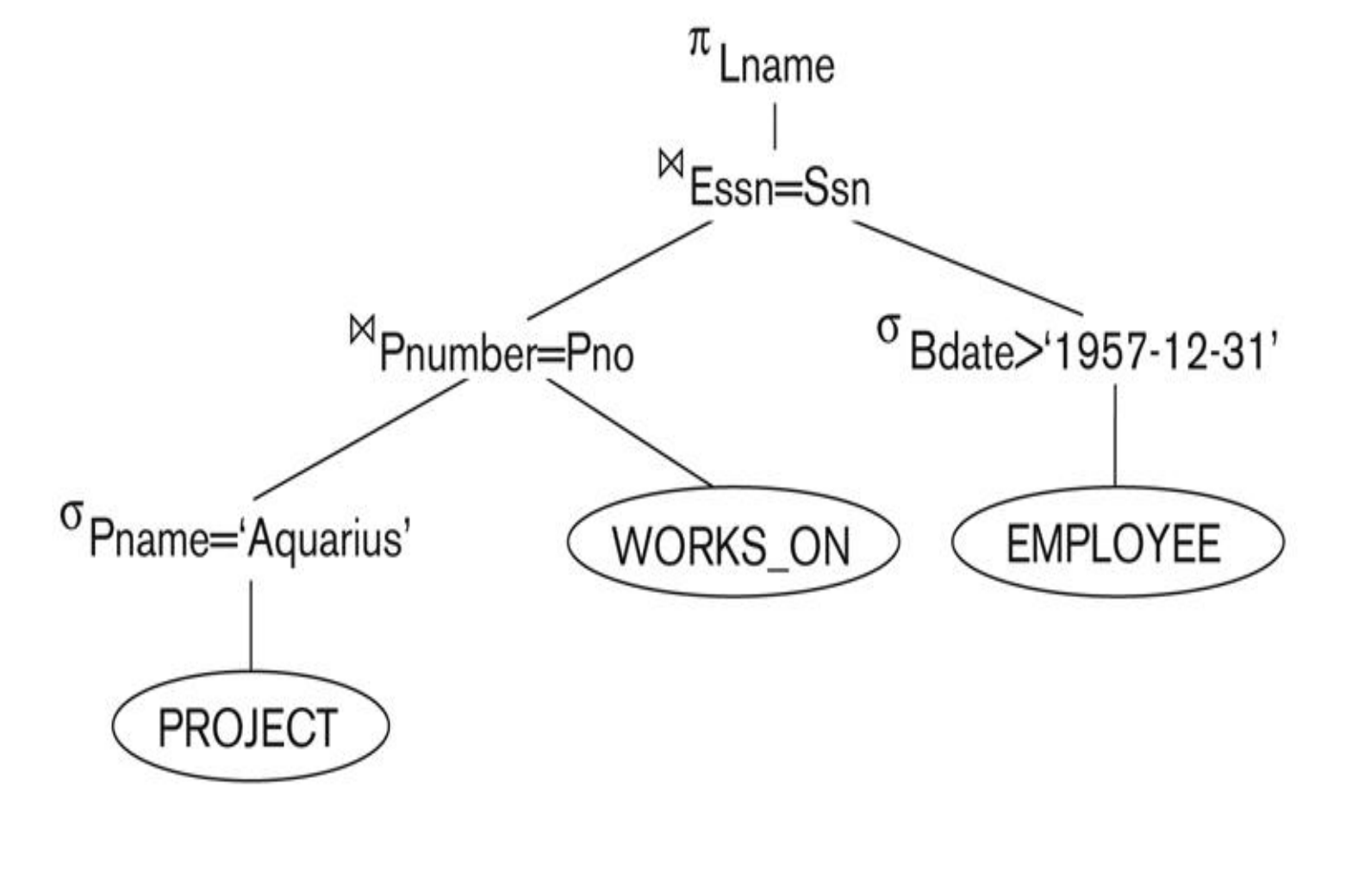
Cont--

Applying the more restrictive select operation first



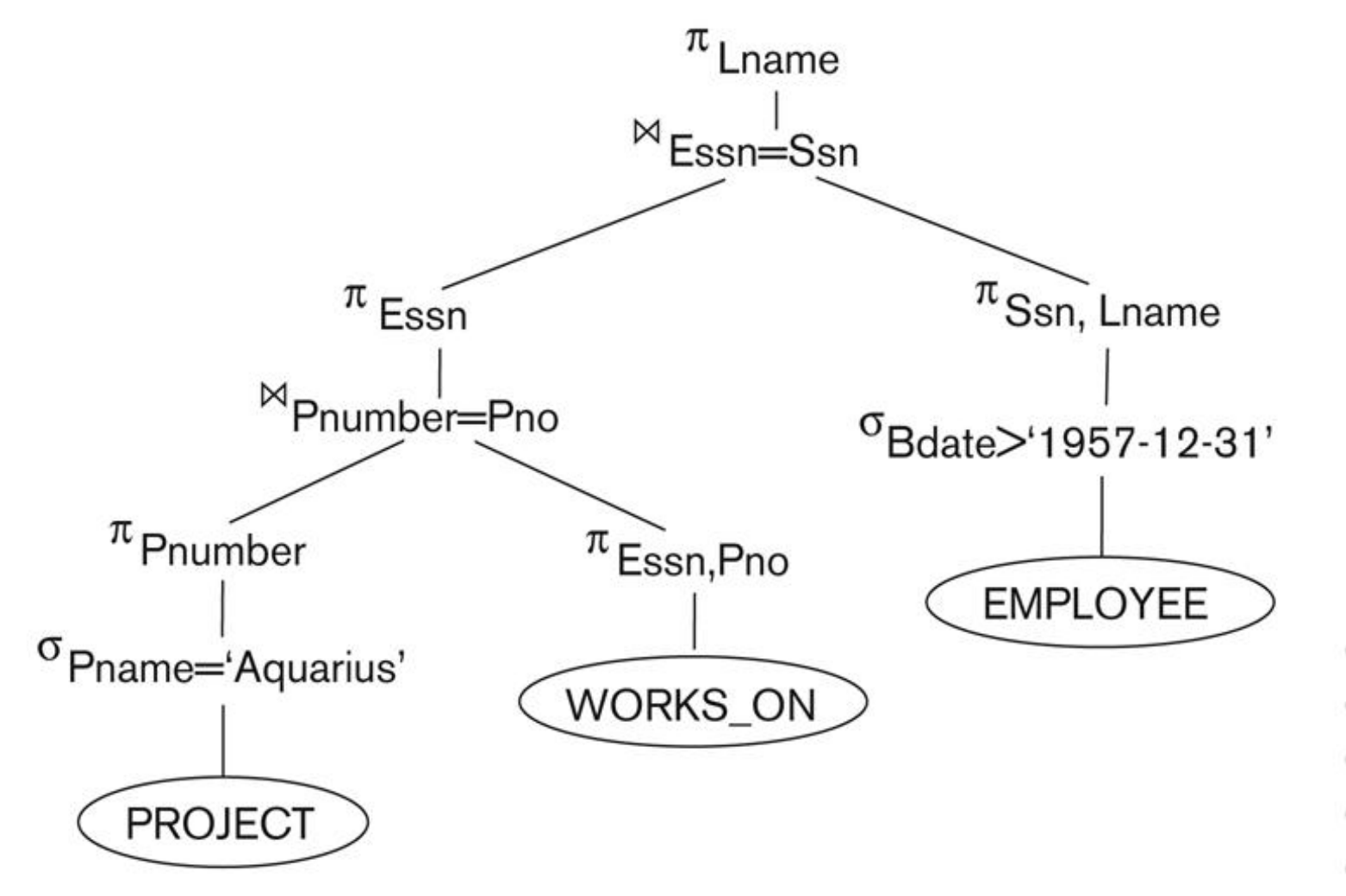
Cont--

Replacing Cartesian product and select with join



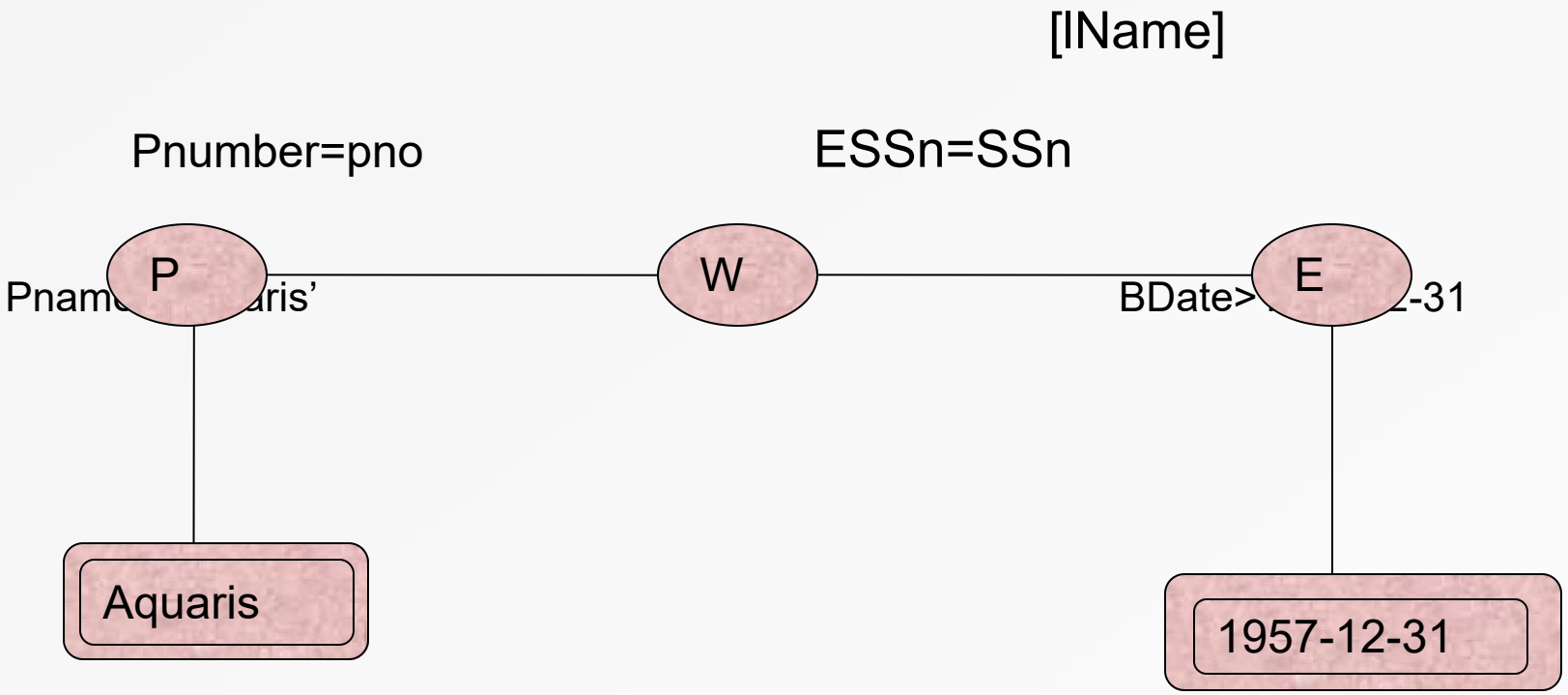
Cont--

Moving project operations down the query tree



Cont…

The query graph representation for queries given in slide 18.



Summary of Heuristic for Algebraic Optimization

- ✓ The same query could correspond to many different relational algebra expressions and hence many different query trees.
- ✓ The main heuristic is to apply first the operations that reduce the size of intermediate results.
- ✓ Perform select operations as early as possible to reduce the number of tuples
- ✓ perfom project operations as early as possible to reduce the number of attributes. (***This is done by moving select and project operations as far down the tree as possible***.)
- ✓ The select and join operations that are most restrictive should be executed before other similar operations.

B. Cost Estimation Approach to Query Optimization

- The main idea is to minimize the cost of processing a query.
- The cost function is comprised of:
  - **I/O** cost+ **CPU** processing cost+ communication cost+ Storage cost
- These components might have different weights in different processing environments
- The DBMs will use information stored in the system catalogue for the purpose of estimating cost.
- The main target of query optimization is to minimize the size of the intermediate relation.
- The size will have effect in the cost of:

1) **Access Cost of Secondary**
2) **StorageStorage Cost**
3) **Computation Cost**
4) **Communication Cost**
5) **Memory usage cost**

Cont--

1. **Access Cost of Secondary Storage**

- Data is going to be accessed from secondary storage. The disk access cost can agin be analyzed in terms of
  - ❖Searching
  - ❖Reading, and
  - ❖Writing, data blocks used to store some portion of a relation.
- **Remark:** The disk access cost will vary depending on
- The file organization used and the access method implemented for the file organization.
- whether the data is stored contiguously or in scattered manner. will affect the disk access cost.

2. **Storage Cost**

- While proce-ssing a query, as any query would be composed of many database operations, there could be one or more intermediate results before reaching the final output. These intermidiate results should be stored in primary memory for further processing.
- The bigger the intermediate relation, the larger the memory requirement,

which will have impact on the limited available space. This will be

considered as acost of storage.

Cont--

3. Computation Cost

- Query is composed of many operations. The operations could be database oprations like reading and writing to disk, or mathematical and other operations like:
  - Searching
  - Sorting
  - Merging
  - Computation on field values

4. Communication Cost

- ln most database systems the database resides in one station and is accessed by various queries originate from different terminals. This will have impact on the performance of the system adding cost for queiy processing. Thus, the cost of transporting data between the database site and the terminal from where the query originate should be analyzed.

5. Memory usage cost

- is the cost pertaining to the number of memory buffers needed during queiy execution.

Large databases

- the access cost to secondary storage is the main emphasis.

Smaller databases

- the emphasis is on minimizing computation cost.

distributed databases

- communication cost must be minimized also.

Group Assignment

1. Select n(number of students in your group) SQL queirs
2. Translate them into a relationla algebra expressions
3. Build query tree for each relational algebra expressions created

N.B: marks depend on how complex your queries are

End